

Guía de lectura activa de la Unidad 2

Administración de bases de datos y programación PL/SQL

Preparación progresiva para INF-324

Construida a partir de las seis presentaciones y cinco guiones SQL de U2

Octubre de 2026

Nombre: _____

Fecha de prueba: Por confirmar

Objetivo de lectura

Al terminar esta guía deberías poder explicar cómo se protege una base de datos, escribir bloques PL/SQL reutilizables, controlar transacciones y concurrencia, y elegir entre una función, una secuencia, un cursor o un trigger según el problema. La meta es comprender cada decisión antes de copiar una solución.

Cómo usarla. Lee una sección breve, tapa el código, reconstruye la idea con tus palabras y responde las preguntas antes de mirar la pauta. En cada módulo marca los errores de concepto, sintaxis y transacción por separado.

Fuentes locales revisadas:

- Presentaciones en U2/presentaciones: Clase 1, Clase 1-2, Clase 2, Clase 2-2, Clase 3 y Clase 4.
- Guiones en U2/codigos: U2_C1.SQL, U2_C1-2.SQL, U2_C2.SQL, U2_C2-2.SQL y U2_C4.SQL.

Índice

1. Mapa de la unidad	3
2. Bloque 1: seguridad, PL/SQL y procedimientos	3
2.1. Lee y explica	3
2.2. Preguntas	4
3. Bloque 2: concurrencia, transacciones y excepciones	4
3.1. Lee y explica	4
3.2. Preguntas	4
4. Bloque 3: funciones PL/SQL	5
4.1. Preguntas	5
5. Bloque 4: secuencias	6
5.1. Preguntas	6
6. Bloque 5: cursores PL/SQL	6
6.1. Preguntas	7
7. Bloque 6: triggers e integridad	7
7.1. Preguntas	7
8. Simulación integradora	8

1. Mapa de la unidad

La Unidad 2 desarrolla el resultado de aprendizaje de administración de bases de datos mediante una secuencia práctica. Las parejas de presentación y código se leen como una sola lección: primero identifica el concepto, luego sigue el orden del script y finalmente modifica el ejemplo.

Bloque	Fuentes	Pregunta guía
1	Clase 1 + U2_C1.SQL	¿Cómo se construye y ejecuta un procedimiento almacenado?
2	Clase 1-2 + U2_C1-2.SQL	¿Cómo se coordinan usuarios, bloqueos, COMMIT, ROLLBACK y excepciones?
3	Clase 2-2 + U2_C2-2.SQL	¿Cuándo conviene una función y qué restricciones tiene?
4	Clase 2 + U2_C2.SQL	¿Cómo una secuencia genera claves sin depender de una tabla?
5	Clase 3	¿Cómo se recorre el resultado de un SELECT con cursores?
6	Clase 4 + U2_C4.SQL	¿Qué evento debe activar un trigger y qué dato debe completar?

Rutina diaria de estudio

Cada sesión dura aproximadamente 40 minutos: 5 para recordar sin mirar, 10 para leer, 20 para escribir o modificar SQL y 5 para corregir. Repite al día siguiente la pregunta en la que obtuviste una respuesta roja. Una semana de práctica puede seguir el orden de los seis bloques y cerrar con una simulación integral.

2. Bloque 1: seguridad, PL/SQL y procedimientos

2.1. Lee y explica

La seguridad de la información se trabaja como confidencialidad, disponibilidad e integridad. En Oracle, PL/SQL agrega variables, estructuras de control y excepciones a SQL. Un bloque tiene las secciones DECLARE, BEGIN, EXCEPTION y END; un procedimiento es un subprograma nombrado, compilado y almacenado que puede recibir parámetros y ejecutar DML.

```

1 CREATE OR REPLACE PROCEDURE CACB_CARGA_USUARIOS(
2   COD_USUARIO_P NUMBER, NOMBRE_USUARIO_P VARCHAR2)
3 IS
4 BEGIN
5   INSERT INTO CACB_USUARIO(COD_USUARIO, NOMBRE_USUARIO)
6   VALUES(COD_USUARIO_P, NOMBRE_USUARIO_P);
7 END;
```

Idea clave

Un procedimiento realiza una acción y puede recibir parámetros; una función debe devolver un valor. No confundas el bloque anónimo que lo invoca con el procedimiento almacenado.

Pausa activa

Sin mirar el ejemplo: explica qué parte declara parámetros, qué parte ejecuta el INSERT y cómo invocarías el procedimiento desde un bloque BEGIN...END.

2.2. Preguntas

1. (Fácil) Nombra los tres objetivos de la seguridad de la información.
2. (Fácil) ¿Qué diferencia hay entre DECLARE y BEGIN?
3. (Fácil) ¿Qué tres operaciones DML se trabajan en los procedimientos de U2?
4. (Intermedia) ¿Por qué CREATE OR REPLACE es útil al practicar?
5. (Intermedia) Escribe un procedimiento que elimine un usuario por código.
6. (Difícil) Diseña un procedimiento I/U/D que reciba una opción y justifica qué parámetros pueden tener valor NULL.

Idea clave

Pauta breve: confidencialidad, disponibilidad e integridad. DECLARE prepara variables y BEGIN ejecuta. DML: INSERT, UPDATE y DELETE. CREATE OR REPLACE recompila y reemplaza la versión anterior. El procedimiento I/U/D debe validar la opción antes de ejecutar la operación.

3. Bloque 2: concurrencia, transacciones y excepciones

3.1. Lee y explica

La concurrencia aparece cuando varios usuarios comparten recursos. LOCK TABLE impone un modo de bloqueo; ROW EXCLUSIVE permite trabajar por filas y evita ciertos bloqueos incompatibles. COMMIT confirma, ROLLBACK deshace y SAVEPOINT ofrece un punto intermedio. Las excepciones se controlan después de BEGIN y pueden producirse con RAISE o RAISE_APPLICATION_ERROR.

```

1 BEGIN
2   LOCK TABLE CACB_USUARIO IN ROW EXCLUSIVE MODE;
3   INSERT INTO CACB_USUARIO VALUES (1, 'CARLOS');
4   COMMIT;
5 EXCEPTION
6   WHEN OTHERS THEN
7     RAISE_APPLICATION_ERROR(-20010, 'ERROR DE CARGA');
8     ROLLBACK;
9 END;
```

Error frecuente

Un SELECT no bloquea una tabla. Además, un bloqueo permanece hasta COMMIT, ROLLBACK o un retroceso a un SAVEPOINT válido. Revisa el flujo de excepción y la posición de cada instrucción.

3.2. Preguntas

1. (Fácil) ¿Qué instrucción confirma una transacción?
2. (Fácil) ¿Qué instrucción deshace el trabajo actual?
3. (Fácil) ¿Qué propósito tiene LOCK TABLE?
4. (Intermedia) Compara ROW SHARE, ROW EXCLUSIVE y EXCLUSIVE.
5. (Intermedia) ¿Por qué una tabla bloqueada aún puede ser consultada?

6. (Difícil) Ordena un procedimiento de carga que bloquee, inserte, confirme y revierta al fallar.

Idea clave

Pauta breve: COMMIT confirma y ROLLBACK deshace. El modo de bloqueo determina qué operaciones simultáneas se permiten. Las consultas no colocan bloqueo de tabla. Un flujo robusto pone LOCK antes del DML, COMMIT al terminar y ROLLBACK en el manejador de excepciones.

4. Bloque 3: funciones PL/SQL

Una función es un bloque nombrado que devuelve un valor y puede usarse dentro de una expresión o consulta. El código de U2 usa `SELECT INTO` para llevar una columna a una variable y captura `NO_DATA_FOUND`. Las funciones llamadas desde SQL no deben ejecutar `INSERT`, `UPDATE` o `DELETE`.

```

1 CREATE OR REPLACE FUNCTION CACB_OBTENER_SUELDO(EMPLOYEE_ID_F NUMBER)
2 RETURN NUMBER IS
3     SUELDO_OBTENIDO NUMBER := 0;
4 BEGIN
5     SELECT SALARY INTO SUELDO_OBTENIDO
6     FROM OEHR_EMPLOYEES WHERE EMPLOYEE_ID = EMPLOYEE_ID_F;
7     RETURN SUELDO_OBTENIDO;
8 EXCEPTION
9     WHEN NO_DATA_FOUND THEN RETURN -1;
10 END;
```

Pausa activa

Tapa el código y explica qué ocurre si el sueldo es NULL y qué ocurre si no existe el empleado. Luego indica cómo llamarías la función desde DUAL.

4.1. Preguntas

1. (Fácil) ¿Qué debe declarar RETURN?
2. (Fácil) ¿Para qué sirve SELECT INTO?
3. (Fácil) ¿Qué excepción indica que no se encontró una fila?
4. (Intermedia) Diferencia una función SQL integrada de una función escrita en PL/SQL.
5. (Intermedia) ¿Por qué una función de consulta no debe ejecutar DML?
6. (Difícil) Diseña una función que devuelva 0 si un valor es NULL y -1 si no existe la fila.

Idea clave

Pauta breve: RETURN declara el tipo y devuelve el valor. `SELECT INTO` asigna el resultado a una variable. `NO_DATA_FOUND` indica ausencia de fila. La función de ejemplo usa IF para transformar NULL en 0 y un manejador para devolver -1.

5. Bloque 4: secuencias

Una secuencia genera enteros únicos para uno o varios usuarios y puede alimentar una clave primaria. Sus números avanzan independientemente de COMMIT o ROLLBACK, por lo que pueden quedar saltos. NEXTVAL incrementa y devuelve el siguiente valor; CURRVAL devuelve el valor actual después de inicializar la secuencia.

```

1 CREATE SEQUENCE CACB_PK_USUARIO
2   START WITH 1 INCREMENT BY 1 MINVALUE 1
3   MAXVALUE 2500 CACHE 200 NOCYCLE;
4
5 INSERT INTO CACB_USUARIO(COD_USUARIO, NOMBRE_USUARIO)
6 VALUES(CACB_PK_USUARIO.NEXTVAL, 'CARLOS');
```

Idea clave

START WITH fija el inicio; INCREMENT BY fija el paso; MINVALUE y MAXVALUE limitan; CYCLE permite reiniciar y NOCYCLE evita reutilizar números.

5.1. Preguntas

1. (Fácil) ¿Qué pseudocolumna se usa primero?
2. (Fácil) ¿Qué diferencia hay entre NEXTVAL y CURRVAL?
3. (Fácil) ¿Por qué una secuencia puede presentar huecos?
4. (Intermedia) Interpreta START WITH 1, INCREMENT BY 1 y MAXVALUE 2500.
5. (Intermedia) ¿Qué efecto tiene NOCYCLE al alcanzar MAXVALUE?
6. (Difícil) Explica por qué una secuencia es preferible a MAX(id)+1 en un procedimiento concurrente.

Idea clave

Pauta breve: NEXTVAL inicializa e incrementa; CURRVAL lee el actual. Los huecos aparecen por consumos concurrentes o transacciones revertidas. NOCYCLE detiene la generación al límite. La secuencia evita colisiones entre sesiones.

6. Bloque 5: cursores PL/SQL

Los cursores permiten manejar el conjunto de filas devuelto por SELECT. Los implícitos los gestiona Oracle para resultados de una fila; los explícitos se declaran y recorren cuando se necesita procesar varias filas o reutilizar la consulta. La declaración del cursor no incluye INTO; el orden se indica con ORDER BY cuando es necesario.

```

1 DECLARE
2   CURSOR c_usuarios IS
3     SELECT COD_USUARIO, NOMBRE_USUARIO
4     FROM CACB_USUARIO ORDER BY COD_USUARIO;
5 BEGIN
6   FOR r IN c_usuarios LOOP
7     DBMS_OUTPUT.PUT_LINE(r.NOMBRE_USUARIO);
8   END LOOP;
9 END;
```

Pausa activa

Dibuja mentalmente el conjunto de filas, el registro `r` y el momento en que el `FOR` abre, obtiene y cierra el cursor.

6.1. Preguntas

1. (Fácil) ¿Qué tipo de cursor se usa para varias filas?
2. (Fácil) ¿Qué representa un cursor internamente?
3. (Fácil) ¿Dónde debe ir `ORDER BY` si importa el orden?
4. (Intermedia) ¿Por qué no se incluye `INTO` en la declaración?
5. (Intermedia) Compara `FOR` `r` `IN` cursor con `OPEN`, `FETCH` y `CLOSE` manuales.
6. (Difícil) Propón un cursor que recorra usuarios y actualice una tabla de auditoría, indicando dónde controlarías errores.

Idea clave

Pauta breve: el cursor explícito maneja varias filas. Es un segmento de memoria asociado al resultado. `INTO` pertenece a la obtención de una fila, no a la declaración. `FOR` gestiona automáticamente apertura, `FETCH` y cierre.

7. Bloque 6: triggers e integridad

Un trigger es un bloque PL/SQL asociado a una tabla que se ejecuta por un evento DML. Sus componentes son nombre, evento, momento y cuerpo; `FOR EACH ROW` lo hace ejecutar por fila. `:NEW` y `:OLD` permiten leer o modificar valores de la fila. Un trigger no puede ejecutar `COMMIT`, `ROLLBACK` ni `SAVEPOINT`.

```

1 CREATE OR REPLACE TRIGGER CACB_PROMEDIO_AUTO
2 BEFORE INSERT ON CACB_DETALLE_NOTAS
3 FOR EACH ROW
4 BEGIN
5     :NEW.PROMEDIO := ROUND((:NEW.NOTA1 + :NEW.NOTA2 + :NEW.NOTA3)/3, 1);
6 END;
```

Error frecuente

No coloques `COMMIT` dentro del trigger. El ejemplo calcula un dato derivado antes del `INSERT`; la transacción la controla el bloque o procedimiento que ejecuta la operación.

7.1. Preguntas

1. (Fácil) ¿Qué operación activa el trigger del ejemplo?
2. (Fácil) ¿Qué representa `:NEW`?
3. (Fácil) ¿Qué cláusula hace que se ejecute por fila?
4. (Intermedia) Diferencia `BEFORE INSERT` de `AFTER INSERT`.
5. (Intermedia) Menciona dos usos de triggers además de calcular promedios.

6. (Difícil) Diseña un trigger que normalice a mayúsculas un nombre antes de insertarlo y explica cómo tratarías NULL.

Idea clave

Pauta breve: el evento es INSERT, :NEW contiene los valores nuevos y FOR EACH ROW ejecuta una vez por fila. Los triggers también mantienen integridad y apoyan auditoría. La normalización puede asignar UPPER(:NEW.nombre) si el valor no es NULL.

8. Simulación integradora

Resuelve sin mirar los módulos y luego revisa cada decisión.

1. Crea una tabla de usuarios con clave primaria.
2. Crea una secuencia para la clave y justifica MAXVALUE, CACHE y NOCYCLE.
3. Escribe un procedimiento de carga que use NEXTVAL, LOCK TABLE, COMMIT y ROLLBACK.
4. Agrega una función que devuelva un dato derivado y trate NO_DATA_FOUND.
5. Declara un cursor explícito para recorrer los usuarios ordenados.
6. Agrega un trigger que complete un campo antes de insertar, sin controlar la transacción.

Habilidad	Intento 1	Intento 2	Intento 3
Procedimiento y bloque PL/SQL			
Bloqueo y transacción			
Excepciones			
Funciones			
Secuencias			
Cursores			
Triggers			

Checklist final

- Puedo explicar confidencialidad, disponibilidad e integridad.
- Puedo distinguir procedimiento, función, secuencia, cursor y trigger.
- Puedo ordenar LOCK, DML, COMMIT y ROLLBACK en una transacción.
- Puedo capturar NO_DATA_FOUND y generar un error propio.
- Puedo explicar NEXTVAL, CURRVAL y los límites de una secuencia.
- Puedo recorrer filas con un cursor y evitar INTO en su declaración.
- Puedo usar :NEW/:OLD y mantener el control de transacciones fuera del trigger.